

Java Backend Interview Series

Sub-Chapter 1 12

java.util.concurrent

Locks . Atomic . Synchronizers . BlockingQueue . Fork/Join

Motivation + Before/After Code + Import Blocks + 30 Q&A + Pitfalls + Tips

1 java.util.concurrent Overview

1.1 Introduction

java.util.concurrent (JUC), introduced in Java 5, provides industrial-strength concurrency utilities above raw synchronized/wait/notify. The package is organized into four areas: the Executor framework (thread pools), concurrent collections (ConcurrentHashMap, BlockingQueue), synchronizers (CountDownLatch, Semaphore, CyclicBarrier), and atomic variables (AtomicInteger, LongAdder).

Area	Key Classes	Purpose
Executor Framework	ExecutorService, ThreadPoolExecutor, ScheduledExecutorService, CompletableFuture	Task submission, thread lifecycle management
Concurrent Collections	ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue impls	Thread-safe collections without external locking
Synchronizers	CountDownLatch, CyclicBarrier, Phaser, Semaphore, Exchanger	Coordinate multiple threads at checkpoints
Atomic Variables	AtomicInteger/Long/Reference, LongAdder, AtomicStampedReference	Lock-free CAS-based atomic compound operations
Locks	ReentrantLock, ReadWriteLock, StampedLock	Flexible locking beyond synchronized

TIP: The JUC library is almost always preferable to raw synchronized/wait/notify. It is better tested, easier to reason about, and exposes richer functionality (tryLock, fairness, Condition, etc.).

2 ReentrantLock Deep Dive

2.1 Introduction

ReentrantLock is the JUC replacement for synchronized. It adds tryLock (non-blocking), lockInterruptibly (cancellable locking), Condition variables (like wait/notifyAll per condition), and optional fairness.

2.2 Basic Usage -- Lock / Unlock in Finally

```
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.TimeUnit;

// ALWAYS unlock in finally to prevent lock leaks
ReentrantLock lock = new ReentrantLock();

lock.lock();

try {
    criticalSection();
} finally {
    lock.unlock(); // guaranteed even if exception thrown
}

// tryLock -- non-blocking: returns false immediately if locked
if (lock.tryLock()) {
    try { doWork(); }
    finally { lock.unlock(); }
} else {
    System.out.println("Lock busy, doing something else");
}

// tryLock with timeout -- avoids deadlock
if (lock.tryLock(500, TimeUnit.MILLISECONDS)) {
    try { doWork(); }
    finally { lock.unlock(); }
} else {
    throw new TimeoutException("Could not acquire lock");
}

// lockInterruptibly -- can be cancelled with Thread.interrupt()
lock.lockInterruptibly();

try { doWork(); }
finally { lock.unlock(); }
```

2.3 Condition Variables -- await / signal / signalAll

```
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.locks.Condition;
import java.util.Queue;
import java.util.LinkedList;

// Condition replaces wait/notifyAll with named, per-condition wait-sets

ReentrantLock lock = new ReentrantLock();
Condition notFull = lock.newCondition();
Condition notEmpty = lock.newCondition();
Queue<String> queue = new LinkedList<>();
final int CAPACITY = 10;

void put(String item) throws InterruptedException {
    lock.lock();
    try {
        while (queue.size() == CAPACITY)
            notFull.await(); // releases lock, waits
        queue.add(item);
        notEmpty.signal(); // wake one consumer
    } finally { lock.unlock(); }
}

String take() throws InterruptedException {
    lock.lock();
    try {
        while (queue.isEmpty())
            notEmpty.await(); // releases lock, waits
        String item = queue.poll();
        notFull.signal(); // wake one producer
        return item;
    } finally { lock.unlock(); }
}
```

Condition variables: await/signal -- two-condition producer-consumer

2.4 Fair vs Non-Fair

```
import java.util.concurrent.locks.ReentrantLock;

// Fair lock: longest-waiting thread gets it next

// Prevents starvation but lower throughput (OS scheduling overhead)
```

```
ReentrantLock fairLock = new ReentrantLock(true); // fair

// Non-fair lock (default): any waiting thread may barge in
// Higher throughput; rare starvation in practice

ReentrantLock defaultLock = new ReentrantLock(); // non-fair

// Inspection

int waitingCount = fairLock.getQueueLength(); // threads waiting for lock

boolean held = fairLock.isLocked();

boolean ownedByMe = fairLock.isHeldByCurrentThread();
```

Fair vs non-fair ReentrantLock

WARNING: ALWAYS place `lock.unlock()` in a `finally` block. If `unlock()` is in the `try` block and an exception occurs before it, the lock is never released, causing all other threads waiting for it to hang indefinitely.

3 ReadWriteLock and StampedLock

3.1 Introduction

ReadWriteLock allows multiple concurrent readers OR one exclusive writer. This improves throughput for read-heavy workloads. StampedLock (Java 8) adds optimistic reads -- even faster, but more complex to use correctly.

3.2 ReentrantReadWriteLock

```
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;
import java.util.concurrent.locks.Lock;
import java.util.HashMap;
import java.util.Map;

ReadWriteLock rwLock = new ReentrantReadWriteLock();

Lock readLock = rwLock.readLock();

Lock writeLock = rwLock.writeLock();

Map<String,String> cache = new HashMap<>();

// READ: multiple threads can hold readLock simultaneously
String get(String key) {
    readLock.lock();

    try {
        return cache.get(key);
    } finally { readLock.unlock(); }
}

// WRITE: exclusive -- no readers or writers may proceed
void put(String key, String value) {
    writeLock.lock();

    try {
        cache.put(key, value);
    } finally { writeLock.unlock(); }
}
```

ReentrantReadWriteLock: concurrent reads, exclusive writes

3.3 StampedLock -- Optimistic Read

```
import java.util.concurrent.locks.StampedLock;

StampedLock sl = new StampedLock();

double x, y;
```

```

// Optimistic read -- NO lock acquired, stamped with a version
double distanceFromOrigin() {
    long stamp = sl.tryOptimisticRead(); // returns non-zero if no writer
    double localX = x, localY = y; // snapshot values
    if (!sl.validate(stamp)) { // writer changed data during read?
        stamp = sl.readLock(); // fall back to real read lock
    }
    try {
        localX = x; localY = y;
    } finally { sl.unlockRead(stamp); }
}

return Math.sqrt(localX*localX + localY*localY);
}

// Write
void move(double deltaX, double deltaY) {
    long stamp = sl.writeLock();
    try {
        x += deltaX; y += deltaY;
    } finally { sl.unlockWrite(stamp); }
}

```

StampedLock: optimistic read with validation fallback

WARNING: StampedLock is NOT reentrant. If a thread holding a write stamp tries to acquire a read lock (or vice versa), it deadlocks. Use StampedLock only when you need maximum read throughput and can manage the added complexity.

TIP: Use ReadWriteLock when reads are much more frequent than writes (e.g., config cache, read-heavy maps). Use StampedLock for extremely high read throughput where the extra optimistic-read complexity is worth it.

4 Atomic Variables and CAS Internals

4.1 Introduction

Atomic variables in `java.util.concurrent.atomic` provide lock-free thread-safe operations via CAS (compare-and-swap) -- a single hardware instruction. They are faster than synchronized for simple operations under low to moderate contention.

4.2 AtomicInteger -- CAS Operations

```
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicReference;

AtomicInteger ai = new AtomicInteger(0);

// Basic ops

int val = ai.get();

ai.set(5);

int old = ai.getAndIncrement(); // returns 0, now 1
int now = ai.incrementAndGet(); // now 2, returns 2
int prev = ai.getAndAdd(10); // returns 2, now 12

// compareAndSet -- atomic conditional update

boolean ok = ai.compareAndSet(12, 0); // if current==12, set to 0
// Returns true and sets to 0. If current != 12, returns false, no change.

// updateAndGet / accumulateAndGet (Java 8)

int result = ai.updateAndGet(n -> n * 2 + 1); // atomic lambda update

// AtomicLong, AtomicBoolean -- same API

AtomicLong al = new AtomicLong(0L);

AtomicBoolean ab = new AtomicBoolean(false);

// AtomicReference -- atomic object reference swap

AtomicReference<String> ref = new AtomicReference<>("v1");

ref.compareAndSet("v1", "v2"); // atomic pointer swap
```

AtomicInteger: CAS operations and updateAndGet

4.3 LongAdder vs AtomicLong Under Contention

```
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.LongAdder;
import java.util.concurrent.atomic.LongAccumulator;
```



```
// AtomicLong: single CAS location -- high contention causes spinning
AtomicLong atomicLong = new AtomicLong(0);
// 1000 threads each calling atomicLong.incrementAndGet()
// Many threads fail CAS and must retry -- "thundering herd"

// LongAdder: distributes increments across per-stripe cells
LongAdder adder = new LongAdder();
adder.increment(); // increments a per-thread stripe cell
adder.add(5);

long total = adder.sum(); // sums all stripe cells (not atomic with add!)
long reset = adder.sumThenReset(); // sum and reset to 0

// LongAccumulator -- generalized LongAdder with custom function
LongAccumulator maxVal = new LongAccumulator(Long::max, Long.MIN_VALUE);
maxVal.accumulate(42);
maxVal.accumulate(17);
long max = maxVal.get(); // 42

// Rule of thumb:
// High write contention, low read frequency -> LongAdder
// Frequent reads of exact value, low contention -> AtomicLong
```

LongAdder vs AtomicLong: high-contention counter

TIP: LongAdder is ideal for metrics counters (requests per second, error counts) where you increment frequently from many threads and read the total only occasionally. For compare-and-swap logic, stick with AtomicLong.

5 CountdownLatch, CyclicBarrier, and Phaser

5.1 CountdownLatch -- One-Shot Barrier

CountDownLatch starts at a count N. Threads call `countDown()` to decrement. Threads waiting via `await()` are released when the count reaches zero. Cannot be reset.

```
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.TimeUnit;

// Parallel initialization pattern

int N = 5;

CountDownLatch ready = new CountDownLatch(N); // workers signal readiness
CountDownLatch start = new CountDownLatch(1); // coordinator signals start
CountDownLatch finish = new CountDownLatch(N); // coordinator waits for done

for (int i = 0; i < N; i++) {
    final int id = i;
    new Thread(() -> {
        initialize(id);
        ready.countDown(); // signal: I am ready
        start.await(); // wait for coordinator to say GO
        doWork(id);
        finish.countDown(); // signal: I am done
    }).start();
}

ready.await(); // wait until all workers are ready
System.out.println("All workers ready -- starting!");
start.countDown(); // release all workers simultaneously
finish.await(30, TimeUnit.SECONDS); // wait max 30s for all to finish
System.out.println("All workers done!");
```

CountDownLatch: parallel initialization and coordinated start

5.2 CyclicBarrier -- Reusable Barrier

CyclicBarrier waits for N threads to reach a common point, then optionally runs a barrier action, then resets for the next cycle. Useful for iterative simulations (e.g., each round all threads must finish before the next round starts).

```
import java.util.concurrent.CyclicBarrier;

int parties = 4;

CyclicBarrier barrier = new CyclicBarrier(parties, () -> {
    // barrierAction: runs ONCE when all parties arrive
})
```

```

System.out.println("Phase complete! Starting next phase.");
});

for (int i = 0; i < parties; i++) {
    final int id = i;
    new Thread(() -> {
        for (int phase = 0; phase < 3; phase++) {
            doPhaseWork(id, phase);
            barrier.await(); // wait for all threads, then continue
        }
    }).start();
}

// barrier automatically resets after each trip -- cyclic!

```

CyclicBarrier: reusable per-phase synchronization

5.3 Phaser -- Flexible Participant Count

```

import java.util.concurrent.Phaser;

// Phaser: like CyclicBarrier but parties can register/deregister dynamically
Phaser phaser = new Phaser(1); // register root party

for (int i = 0; i < 3; i++) {
    phaser.register(); // register each worker
    final int id = i;
    new Thread(() -> {
        for (int phase = 0; phase < 3; phase++) {
            doWork(id);
            phaser.arriveAndAwaitAdvance(); // wait for all to arrive
        }
        phaser.arriveAndDeregister(); // done -- leave the phaser
    }).start();
}

// Root party controls phases
for (int phase = 0; phase < 3; phase++) {
    phaser.arriveAndAwaitAdvance(); // wait for phase completion
    System.out.println("Phase " + phase + " complete");
}

```

Phaser: dynamic party registration for multi-phase tasks

Synchronizer	Reusable?	Participants	Use Case
CountDownLatch	No (one-shot)	Any number of waiters	Service startup readiness, parallel test init
CyclicBarrier	Yes (auto-reset)	Fixed N at creation	Iterative simulation, multi-phase algorithms
Phaser	Yes (per-phase)	Dynamic (register/deregister)	Variable-participant multi-phase tasks

6 Semaphore -- Rate Limiting and Resource Pooling

6.1 Introduction

A Semaphore maintains a set of permits. `acquire()` blocks until a permit is available. `release()` returns a permit. Semaphore is ideal for limiting concurrent access to a resource (connection pool, rate limiter).

```
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;

// Connection pool pattern: limit concurrent DB connections
Semaphore pool = new Semaphore(10); // max 10 concurrent connections

Connection getConnection() throws InterruptedException {
    pool.acquire(); // block until a slot is free
    try {
        return openConnection();
    } catch (Exception e) {
        pool.release(); // release if openConnection fails
        throw e;
    }
}

void releaseConnection(Connection c) {
    closeConnection(c);
    pool.release(); // return the permit
}

// tryAcquire -- non-blocking
if (pool.tryAcquire()) {
    try { doWork(); }
    finally { pool.release(); }
} else {
    throw new ServiceUnavailableException("Pool exhausted");
}

// tryAcquire with timeout
if (pool.tryAcquire(1, TimeUnit.SECONDS)) { ... }

// Rate limiting: allow max N concurrent API calls
Semaphore rateLimiter = new Semaphore(5, true); // fair
```

Semaphore: connection pool and rate limiting

TIP: A Semaphore with 1 permit acts like a mutex, but unlike synchronized it is not reentrant -- the same thread can deadlock itself by calling `acquire()` twice without `release()`. Use `ReentrantLock` for mutual exclusion.

7 BlockingQueue Implementations

7.1 Introduction

BlockingQueue is a queue that blocks on put() when full and blocks on take() when empty. It is the standard way to implement producer-consumer without explicit wait/notify. Choose the implementation based on capacity, ordering, and throughput requirements.

Implementation	Capacity	Order	Key Property	Use Case
ArrayBlockingQueue	Bounded (fixed)	FIFO	Single lock; fair option	Bounded task queue; back-pressure
LinkedBlockingQueue	Bounded or unbounded	FIFO	Separate put/take locks	Higher throughput than array impl
PriorityBlockingQueue	Unbounded	Priority order	Natural order or Comparator	Task scheduling by priority
SynchronousQueue	Zero (no buffer)	Direct hand-off	put() blocks until take() called	Handoff; newCachedThreadPool
DelayQueue	Unbounded	Delay expiry order	take() blocks until element delay expires	Scheduled tasks, TTL cache eviction
LinkedTransferQueue	Unbounded	FIFO	transfer() blocks until consumed	High-throughput producer-consumer

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.TimeUnit;

// Producer-consumer with BlockingQueue -- no explicit wait/notify needed
BlockingQueue<String> queue = new ArrayBlockingQueue<>(100);

// Producer
Runnable producer = () -> {
    while (!done) {
        String item = produceItem();
        queue.put(item); // blocks if queue is full (back-pressure)
    }
};

// Consumer
Runnable consumer = () -> {
    while (!done) {
        String item = queue.poll(100, TimeUnit.MILLISECONDS);
```

```
if (item != null) processItem(item);
}
};

// offer() -- non-blocking: returns false if full
boolean added = queue.offer(item);

// offer with timeout
boolean added2 = queue.offer(item, 500, TimeUnit.MILLISECONDS);
```

BlockingQueue producer-consumer pattern

WARNING: `LinkedBlockingQueue` created without a capacity argument is UNBOUNDED (`Integer.MAX_VALUE`). This is the same risk as `Executors.newFixedThreadPool` -- a slow consumer can cause OOM. Always specify a capacity in production.

8 Fork/Join Framework

8.1 Introduction

The Fork/Join framework (Java 7) executes recursive divide-and-conquer algorithms on `ForkJoinPool`. It uses work-stealing: idle worker threads steal tasks from the queues of busy workers, maximizing CPU utilization. `ForkJoinPool.commonPool()` is shared by parallel streams, `CompletableFuture` defaults, and fork/join tasks.

8.2 RecursiveTask -- Divide and Conquer Sum

```
import java.util.concurrent.RecursiveTask;
import java.util.concurrent.ForkJoinPool;
import java.util.stream.LongStream;

class SumTask extends RecursiveTask<Long> {

    private final long[] arr;
    private final int lo, hi;
    private static final int THRESHOLD = 1000;

    SumTask(long[] arr, int lo, int hi) {
        this.arr = arr; this.lo = lo; this.hi = hi;
    }

    @Override
    protected Long compute() {
        if (hi - lo <= THRESHOLD) {
            // Base case: compute sequentially
            long sum = 0;
            for (int i = lo; i < hi; i++) sum += arr[i];
            return sum;
        }
        // Fork: split in half
        int mid = (lo + hi) / 2;
        SumTask left = new SumTask(arr, lo, mid);
        SumTask right = new SumTask(arr, mid, hi);
        left.fork(); // async: submit left task
        long rightResult = right.compute(); // run right in current thread
        long leftResult = left.join(); // wait for left result
        return leftResult + rightResult;
    }
}
```

```
// Run in the common pool

long[] data = LongStream.range(0, 10_000_000).toArray();

ForkJoinPool pool = ForkJoinPool.commonPool();

long sum = pool.invoke(new SumTask(data, 0, data.length));

System.out.println("Sum: " + sum);
```

RecursiveTask: parallel array sum with fork/join

8.3 RecursiveAction -- Parallel Array Fill

```
import java.util.concurrent.RecursiveAction;
import java.util.concurrent.ForkJoinPool;
import java.util.Arrays;

class FillTask extends RecursiveAction {
    private final int[] arr;
    private final int lo, hi, value;
    private static final int THRESHOLD = 1000;

    FillTask(int[] arr, int lo, int hi, int value) {
        this.arr=arr; this.lo=lo; this.hi=hi; this.value=value;
    }

    @Override
    protected void compute() {
        if (hi - lo <= THRESHOLD) {
            Arrays.fill(arr, lo, hi, value);
            return;
        }

        int mid = (lo + hi) / 2;
        invokeAll(new FillTask(arr, lo, mid, value),
            new FillTask(arr, mid, hi, value));
    }
}

int[] data = new int[10_000_000];
new ForkJoinPool().invoke(new FillTask(data, 0, data.length, 99));
```

RecursiveAction: parallel array fill with invokeAll

TIP: Choose the THRESHOLD carefully: too small means too many tasks (overhead > gain); too large means tasks are not split enough (under-utilization). A common starting point: `THRESHOLD = array.length / (4 * Runtime.getRuntime().availableProcessors())`.

9 CompletableFuture -- Advanced Patterns

9.1 Introduction

CompletableFuture (Java 8) enables non-blocking async pipelines. Key design points for production: always supply a custom thread pool to async methods, understand the difference between `thenApply` (same thread) and `thenApplyAsync` (different thread), and handle exceptions explicitly.

9.2 Custom Thread Pool and Async vs Sync Callbacks

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

// Named thread pool for CompletableFuture chains
ExecutorService io = Executors.newFixedThreadPool(20, r -> {
    Thread t = new Thread(r);
    t.setName("cf-io-" + t.getId());
    t.setDaemon(true);
    return t;
});

// thenApply: runs on the COMPLETING thread (may be the caller)
CompletableFuture<Integer> cf1 = CompletableFuture
    .supplyAsync(() -> fetchString(), io) // runs on io pool
    .thenApply(String::length); // runs on io pool thread

// thenApplyAsync: always submits to a specified pool
CompletableFuture<Integer> cf2 = CompletableFuture
    .supplyAsync(() -> fetchString(), io)
    .thenApplyAsync(String::length, io); // runs on io pool thread too

// Complex async pipeline
CompletableFuture<String> result = CompletableFuture
    .supplyAsync(() -> fetchUserId(), io)
    .thenComposeAsync(id -> fetchUser(id), io)
    .thenApplyAsync(user -> enrichUser(user), io)
    .exceptionally(ex -> fallbackUser())
    .whenComplete((user, ex) -> logResult(user, ex));
```

CompletableFuture: custom executor and async pipeline

9.3 Combining Multiple Futures

```
import java.util.concurrent.CompletableFuture;
import java.util.List;

// allOf -- wait for ALL, then extract results
CompletableFuture<User> userFuture = fetchUserAsync(1L);
CompletableFuture<Account> accountFuture = fetchAccountAsync(1L);
CompletableFuture<List<Order>> ordersFuture = fetchOrdersAsync(1L);

CompletableFuture<Void> all = CompletableFuture
    .allOf(userFuture, accountFuture, ordersFuture);

CompletableFuture<Dashboard> dashboard = all.thenApply(v -> {
    // all futures are done here -- join() won't block
    User user = userFuture.join();
    Account account = accountFuture.join();
    List<Order> orders = ordersFuture.join();
    return buildDashboard(user, account, orders);
});

// anyOf -- first to complete wins (result is Object)
CompletableFuture<Object> first = CompletableFuture
    .anyOf(primaryCache, secondaryCache, dbFallback);

// thenCombine -- zip exactly two futures
CompletableFuture<String> combined =
    userFuture.thenCombine(accountFuture,
        (u, a) -> u.getName() + " / " + a.getBalance());
```

allOf, anyOf, thenCombine for combining futures

WARNING: `CompletableFuture.get()` throws `ExecutionException` wrapping the real exception. Prefer `join()` in streams/lambda (throws `CompletionException` -- unchecked). Always add `.exceptionally()` or `.handle()` to prevent silent failures.

10 ConcurrentHashMap -- Advanced Operations

10.1 Introduction

ConcurrentHashMap (CHM) is the go-to thread-safe map. Unlike Hashtable (fully synchronized) or Collections.synchronizedMap() (single lock), CHM uses segment-level (Java 7) or node-level (Java 8+) CAS locking for dramatically higher throughput under concurrent access.

10.2 Atomic Compound Operations

```
import java.util.concurrent.ConcurrentHashMap;

ConcurrentHashMap<String,Integer> map = new ConcurrentHashMap<>();

// computeIfAbsent -- atomic: put only if key absent
// NEVER race condition here unlike: if(!containsKey) put()
map.computeIfAbsent("key", k -> expensiveCompute(k));

// computeIfPresent -- atomic: update only if key present
map.computeIfPresent("key", (k, v) -> v + 1);

// compute -- atomic: create OR update (merge logic)
map.compute("key", (k, v) -> v == null ? 1 : v + 1);

// merge -- combine old and new value
map.merge("key", 1, Integer::sum); // increment counter, initialize to 1

// Bulk parallel operations (Java 8)
map.forEach(2, (k, v) -> // parallelismThreshold=2
System.out.println(k + "=" + v));

long count = map.mappingCount(); // use this instead of size() for large maps

// putIfAbsent -- returns existing value or null if newly inserted
Integer existing = map.putIfAbsent("newKey", 100);
```

ConcurrentHashMap: atomic compound operations

TIP: Use `map.merge(key, 1, Integer::sum)` to implement thread-safe word-count or event-count maps. This is both simpler and faster than get-check-put patterns because merge is an atomic compound operation in ConcurrentHashMap.

11 Interview Q and A -- java.util.concurrent

30 questions covering standard and advanced depth. [ADV] marks senior-level questions.

Q: What are the main areas of java.util.concurrent?

A: Executor framework (thread pools, Futures), concurrent collections (ConcurrentHashMap, BlockingQueue), synchronizers (CountDownLatch, Semaphore, CyclicBarrier), atomic variables (AtomicInteger, LongAdder), and locks (ReentrantLock, ReadWriteLock, StampedLock).

Q: Why prefer ReentrantLock over synchronized?

A: ReentrantLock adds: tryLock (non-blocking), tryLock(timeout) (avoids deadlock), lockInterruptibly (cancellable), multiple Condition variables per lock, fairness option, and introspection (isLocked(), getQueueLength()). synchronized is simpler -- use it when you don't need these extras.

Q: What must always happen with ReentrantLock.unlock()?

A: It must be called in a finally block. If the try block throws before unlock(), the lock leaks forever and all threads waiting for it hang indefinitely. Never place unlock() in the try body -- it may be skipped on exceptions.

Q: What does ReentrantLock.tryLock() do?

A: Attempts to acquire the lock without blocking. Returns true immediately if the lock is available, false if not. Optional timeout variant: tryLock(long timeout, TimeUnit unit) blocks for at most timeout before returning false.

Q: What is a Condition variable?

A: An object obtained from lock.newCondition() that provides await()/signal()/signalAll() -- like wait/notify but scoped to a specific condition. A single ReentrantLock can have multiple Conditions (e.g., notFull and notEmpty for a bounded buffer).

Q: What is ReadWriteLock and when should you use it?

A: Allows multiple concurrent readers OR one exclusive writer. Use when reads are much more frequent than writes (e.g., config cache, read-heavy lookup table). Writers block all readers and other writers; readers block only writers.

Q: What is StampedLock optimistic read?

A: tryOptimisticRead() returns a stamp without acquiring any lock. After reading shared data, validate(stamp) checks if a writer changed the data. If invalidated, fall back to a real readLock. Optimistic reads are faster than read locks when writes are rare.

Q: What is the difference between AtomicInteger and synchronized int?

A: AtomicInteger uses CAS (hardware compare-and-swap) which is a single instruction -- no OS-level lock, no thread suspension. synchronized acquires a JVM monitor which may involve OS thread scheduling. Under low to moderate contention, AtomicInteger is faster. Under very high contention, LongAdder is better.

Q: What does compareAndSet() do in AtomicInteger?

A: Atomically: if current value == expected, set to newValue and return true. Otherwise return false without change. This is the CAS primitive that all atomic operations are built on. It maps to a single hardware instruction (e.g., CMPXCHG on x86).

Q: When should you use LongAdder over AtomicLong?

A: Under high write contention (many threads incrementing frequently). LongAdder distributes increments across per-stripe cells, reducing CAS contention. Use AtomicLong when you need compareAndSet logic or when reads are as frequent as writes.

Q: What is CountDownLatch?

A: A one-shot synchronizer: initialized with count N. Threads call countdown() to decrement; threads call await() to block until count reaches zero. Cannot be reset. Use for: waiting for N services to start, waiting for N parallel

tasks to complete.

Q: Difference between CountdownLatch and CyclicBarrier?

A: CountdownLatch: one-shot (cannot reset), any thread can countDown, used for 'wait for N events'.
CyclicBarrier: reusable (auto-resets after each trip), exactly N threads must reach the barrier, optional barrier action runs once all arrive. Use CyclicBarrier for repeated phase-synchronization.

Q: What is Phaser and how does it differ from CyclicBarrier?

A: Phaser supports dynamic participant count (register()/deregister()), multiple phases, and termination.
CyclicBarrier has a fixed participant count set at creation. Phaser is more flexible for algorithms where workers join and leave across phases.

Q: What does Semaphore do?

A: Maintains N permits. acquire() blocks until a permit is available, then takes one. release() returns one. Use for: limiting concurrent access to a resource (connection pool), rate limiting, or as a mutex (Semaphore(1), but not reentrant).

Q: What BlockingQueue implementation should you use for a producer-consumer buffer?

A: ArrayBlockingQueue(capacity) for a bounded buffer with predictable memory use.
LinkedBlockingQueue(capacity) for slightly higher throughput (separate locks for put/take). Always specify capacity to avoid OOM from unbounded growth.

Q: What is SynchronousQueue?

A: A BlockingQueue with zero capacity. put() blocks until another thread calls take(), and vice versa. It is a direct hand-off mechanism with no buffering. Used internally by Executors.newCachedThreadPool() to hand tasks directly to idle threads.

Q: What is the Fork/Join framework?

A: An ExecutorService (ForkJoinPool) designed for divide-and-conquer recursive tasks. Workers use work-stealing: idle threads steal tasks from busy thread queues. RecursiveTask<V> returns a value; RecursiveAction returns void. ForkJoinPool.commonPool() is shared by parallel streams.

Q: What is work-stealing in ForkJoinPool?

A: Each worker thread has its own deque of tasks. When a thread's queue is empty, it steals tasks from the tail of another busy thread's deque. This minimizes idle time and maximizes CPU utilization for recursive divide-and-conquer workloads.

Q: What is the difference between fork() and invoke() in Fork/Join?

A: fork() asynchronously submits a task to the pool and returns immediately. join() waits for the forked task's result. invokeAll() forks multiple tasks and waits for all. invoke() is a synchronous submit-and-wait from outside the pool. The pattern is: fork left subtask, compute right in current thread, join left.

Q: Why should you always use thenApplyAsync over thenApply in production?

A: thenApply runs on the completing thread (which may be an I/O thread or ForkJoinPool worker). If the callback is slow, it blocks that thread. thenApplyAsync submits to a dedicated Executor, preventing one slow callback from starving other completions. Use a custom Executor -- not ForkJoinPool.commonPool() -- for I/O callbacks.

Q: [ADV] Explain ConcurrentHashMap's internal lock granularity in Java 8.

A: Java 8 CHM removed the segment array. Each hash bucket (array slot) uses CAS for insertion of the first entry. Bucket-level synchronized blocks are used only when chaining in an existing bucket. This dramatically reduces contention -- most operations need no lock at all, just a CAS on an individual node.

Q: [ADV] What happens when ConcurrentHashMap transitions from list to tree?

A: When a bucket's chain exceeds TREEIFY_THRESHOLD (8), and the overall map size exceeds MIN_TREEIFY_CAPACITY (64), the chain is converted to a TreeNode (red-black tree). This keeps worst-case per-bucket lookup O(log n) instead of O(n) under hash collisions (e.g., from a bad hashCode or deliberate attacks).

Q: [ADV] Explain ABA problem with AtomicReference.compareAndSet()

A: Thread A reads value=A, Thread B changes to B then back to A, Thread A's CAS succeeds -- but the object changed and changed back. The value looks the same but the meaning is different (e.g., a node in a lock-free stack was removed and reinserted). Fix: AtomicStampedReference adds a version stamp; CAS checks both value AND stamp.

Q: [ADV] What is the difference between Condition.await() and Object.wait()?

A: Both release the lock and wait. Condition allows multiple wait-sets per lock (e.g., notFull and notEmpty), so you can wake specifically the right set of threads. Object.wait() has only one wait-set per monitor -- notifyAll() wakes ALL waiting threads, potentially waking threads waiting for a different condition.

Q: [ADV] How does StampedLock prevent lock upgrade from optimistic read to write lock?

A: StampedLock has no direct tryConvertToWriteLock upgrade path that is safe in all situations. The stamp from tryOptimisticRead() is always invalidated by any write. To upgrade: release the optimistic stamp, acquire writeLock(). Attempting to upgrade while holding a readLock can deadlock since readLock and writeLock are mutually exclusive.

Q: [ADV] What is the parallelism parameter in ForkJoinPool and ForkJoinPool.commonPool()?

A: ForkJoinPool(int parallelism) creates a pool with that many worker threads. ForkJoinPool.commonPool() uses Runtime.getRuntime().availableProcessors() - 1 (leaving one core for the calling thread). You can override commonPool parallelism with -Djava.util.concurrent.ForkJoinPool.common.parallelism=N.

Q: [ADV] What is a phased countdown pattern with CountdownLatch?

A: Use two CountdownLatches: one (ready) counted down by workers when initialized; one (start) counted down by the coordinator to release workers simultaneously. This enables a coordinated starting gun: all workers block on start.await(), coordinator blocks on ready.await(), then triggers all workers by counting down start.

Q: [ADV] Explain CompletableFuture.allOf() limitation and how to fix it.

A: allOf() returns CompletableFuture<Void> -- it doesn't aggregate individual results. After allOf completes, you must call join() on each original CF individually. For a clean API: CompletableFuture.allOf(futures).thenApply(v -> Stream.of(futures).map(CompletableFuture::join).collect(Collectors.toList())).

Q: [ADV] What is DelayQueue and how does it work?

A: DelayQueue<E extends Delayed> holds Delayed elements. take() blocks until an element's getDelay() returns <= 0. Internally uses a PriorityQueue sorted by delay. Use for: scheduled retries, TTL-based cache eviction, task scheduling. The Delayed interface's compareTo() must be consistent with getDelay().

Q: [ADV] What makes ConcurrentHashMap.compute() atomic?

A: compute(key, remappingFunction) applies the function and updates the map atomically under a bucket-level lock. No other thread can update the same key between the read and the write. This is why compute() is safe for incrementing counters: it replaces the unsafe get-check-put pattern with a single atomic step.

12 Summary -- java.util.concurrent Cheat Sheet

Utility	Key API	When to Use
ReentrantLock	lock/unlock (finally!), tryLock, Condition	When synchronized is not enough: timeouts, multiple conditions
ReadWriteLock	readLock(), writeLock()	Read-heavy shared state (config cache, lookup maps)
StampedLock	tryOptimisticRead(), validate(), writeLock()	Maximum read throughput; not reentrant; complex to use
AtomicInteger	getAndIncrement(), compareAndSet()	Low-contention atomic counter or CAS logic
LongAdder	increment(), sum()	High-contention write-heavy counter (metrics)
CountDownLatch	countDown(), await()	Wait for N events; one-shot (not resettable)
CyclicBarrier	await(), optional barrierAction	Repeated phase synchronization for fixed N threads
Semaphore	acquire(), release(), tryAcquire()	Resource pool, rate limiter, concurrency cap
BlockingQueue	put() / take() / offer() / poll()	Producer-consumer; always use bounded capacity
ForkJoinPool	RecursiveTask.fork()/join(), invokeAll()	Recursive divide-and-conquer; work-stealing
CompletableFuture	supplyAsync, thenCompose, handle, allOf	Non-blocking async chains; always use custom Executor
ConcurrentHashMap	computeIfAbsent, merge, compute	Thread-safe map; atomic compound operations